

DATABASE SECURITY SETUP

(Roles, Network Policy, and Data Masking)

1. CREATE AND MANAGE USERS AND ROLES

Step 1: Create Roles

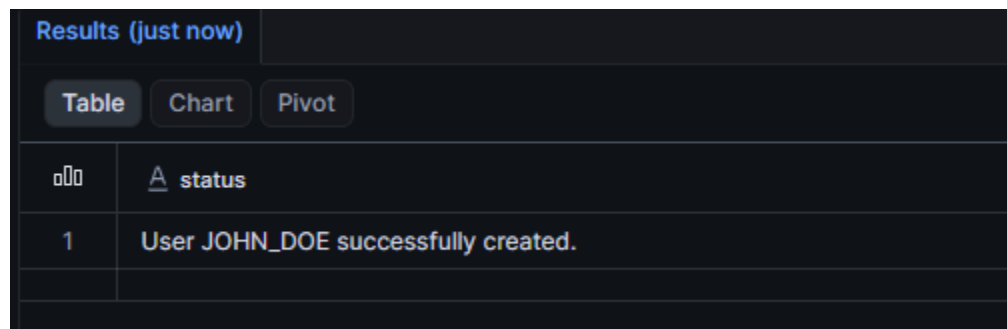
```
CREATE ROLE admin;  
CREATE ROLE analyst;  
CREATE ROLE viewer;
```

Step 2: Assign Privileges to Roles

```
GRANT ALL PRIVILEGES ON DATABASE sales_db TO ROLE admin;  
GRANT SELECT ON ALL TABLES IN SCHEMA sales_db.public TO ROLE analyst;  
GRANT USAGE ON WAREHOUSE compute_wh TO ROLE viewer;
```

Step 3: Create Users

```
CREATE USER john_doe  
PASSWORD = 'Password123!'  
DEFAULT_ROLE = analyst  
MUST_CHANGE_PASSWORD = TRUE;
```

A screenshot of a database management interface. At the top, there's a tab labeled "Results (just now)". Below it are three buttons: "Table", "Chart", and "Pivot". The "Table" button is selected. The table has two columns: an icon column and a "status" column. The first row shows a checkmark icon and the text "User JOHN_DOE successfully created.".

Results (just now)	
Table Chart Pivot	
🔍	status
1	User JOHN_DOE successfully created.

Step 4: Assign Roles to Users

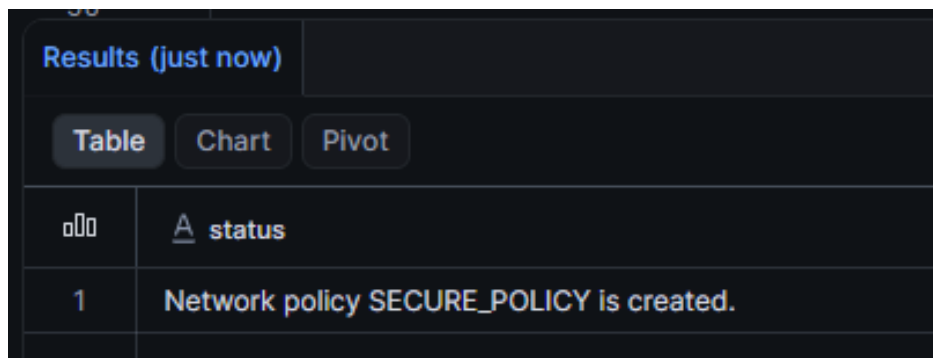
```
GRANT ROLE admin TO USER admin_user;
```

```
GRANT ROLE analyst TO USER john_doe;  
GRANT ROLE viewer TO USER jane_doe;
```

2. CONFIGURE NETWORK POLICIES

Step 1: Create a Network Policy

```
CREATE NETWORK POLICY secure_policy  
ALLOWED_IP_LIST = ('192.168.1.0/24', '203.0.113.0/32')  
BLOCKED_IP_LIST = ('10.0.0.0/8');
```



The screenshot shows a database query results window. At the top, it says "Results (just now)". Below this are three tabs: "Table", "Chart", and "Pivot". The "Table" tab is selected. The table has two columns: an icon column and a "status" column. The first row shows the number "1" in the icon column and the text "Network policy SECURE_POLICY is created." in the status column.

	status
1	Network policy SECURE_POLICY is created.

Step 2: Apply Network Policy

```
ALTER ACCOUNT SET NETWORK_POLICY = secure_policy;
```

3. IMPLEMENT DATA MASKING ON SENSITIVE DATA

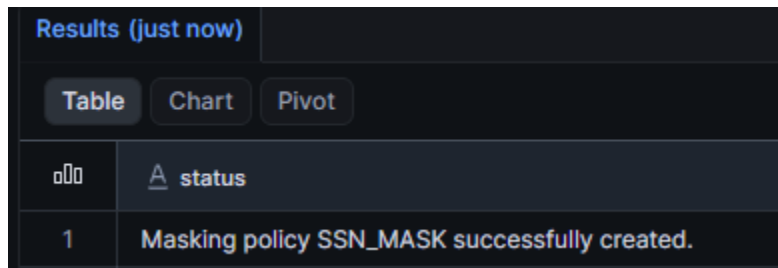
Step 1: Create a Table with Sensitive Data

```
CREATE TABLE employees (  
  id INT,  
  name STRING,  
  ssn STRING  
);
```

```
INSERT INTO employees VALUES (1, 'Alice', '123-45-6789');
```

Step 2: Define a Masking Policy

```
CREATE MASKING POLICY ssn_mask AS (val STRING) RETURNS STRING ->  
CASE  
  WHEN CURRENT_ROLE() = 'admin' THEN val  
  ELSE 'XXX-XX-XXXX'  
END;
```



The screenshot shows a database interface with a 'Results (just now)' header. Below the header are three tabs: 'Table', 'Chart', and 'Pivot'. The 'Table' tab is selected. The table has one row with the text 'Masking policy SSN_MASK successfully created.'.

	status
1	Masking policy SSN_MASK successfully created.

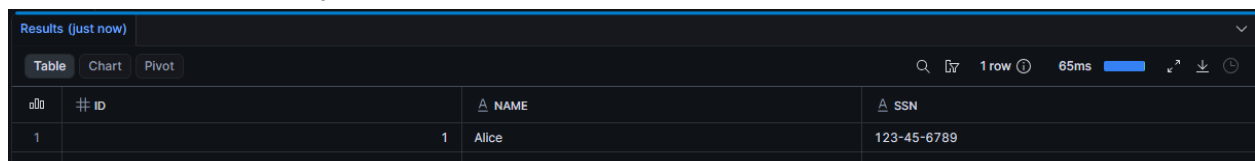
Step 3: Apply the Masking Policy

```
ALTER TABLE employees  
MODIFY COLUMN ssn  
SET MASKING POLICY ssn_mask;
```

Step 4: Test the Masking Policy

-- Admin role (unmasked data)

```
SET ROLE admin;  
SELECT * FROM employees;
```

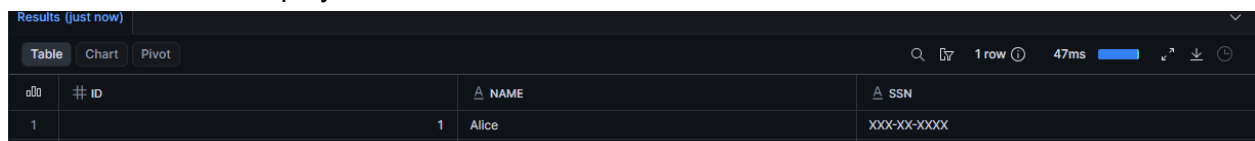


The screenshot shows a database interface with a 'Results (just now)' header. Below the header are three tabs: 'Table', 'Chart', and 'Pivot'. The 'Table' tab is selected. The table has three columns: '# ID', 'NAME', and 'SSN'. The first row shows the data for Alice, with her SSN as '123-45-6789'.

# ID	NAME	SSN
1	Alice	123-45-6789

-- Analyst role (masked data)

```
SET ROLE analyst;  
SELECT * FROM employees;
```



The screenshot shows a database interface with a 'Results (just now)' header. Below the header are three tabs: 'Table', 'Chart', and 'Pivot'. The 'Table' tab is selected. The table has three columns: '# ID', 'NAME', and 'SSN'. The first row shows the data for Alice, with her SSN masked as 'XXX-XX-XXXX'.

# ID	NAME	SSN
1	Alice	XXX-XX-XXXX